

SkillCluster AI

AI-Powered Resume Clustering and Skill Gap Detection System

Project Report

Project Title	SkillCluster AI: Resume Clustering & Skill Gap Detection using NLP and Machine Learning
Submitted By	Shubham Kumar, Akshat Singh, Shubham Sharma
Branch / Department	Computer Science & Engineering
Academic Year	2025–2026
Technologies Used	Python, NLP (NLTK), Scikit-Learn, TF-IDF, K-Means, Streamlit, Plotly
GitHub Repository	github.com/shubhamgautam05/SkillCluster_AI

Academic Year 2025–2026

1. Abstract

The increasing volume of student resumes processed during campus placements poses a significant challenge for placement cells in educational institutions. Manual evaluation is time-consuming, subjective, and incapable of providing actionable feedback at scale. This project, SkillCluster AI, proposes an automated, AI-powered system that leverages Natural Language Processing (NLP) and Unsupervised Machine Learning to analyze, cluster, and evaluate student resumes.

The system ingests raw, unstructured resume text; applies text preprocessing techniques including tokenization, stopword removal, and lemmatization; extracts structured technical skills using a rule-based NLP engine; generates numerical representations via TF-IDF (Term Frequency–Inverse Document Frequency) vectorization; and groups candidates into distinct proficiency clusters using the K-Means clustering algorithm. A comparative analysis between the strongest and weakest clusters identifies critical skill gaps, and the system generates personalized learning path recommendations to help students bridge those gaps.

A modern, interactive dashboard built with Streamlit and Plotly provides real-time visualization of cluster distributions, skill profiles, and a live resume analyzer for individual candidate assessment.

Keywords: *NLP, K-Means Clustering, TF-IDF, Skill Gap Detection, Resume Analysis, Unsupervised Learning, Streamlit.*

2. Introduction

2.1 Background

In the current competitive job market, the ability to quickly assess and categorize candidate resumes is crucial for recruitment teams and academic placement cells. Traditional manual screening suffers from inherent limitations:

- **Scalability Issues:** A placement cell handling 500+ resumes cannot provide individual skill feedback to each student.
- **Subjectivity:** Human reviewers introduce bias and inconsistency in evaluation.
- **Lack of Actionable Insights:** Students rarely receive concrete, data-driven feedback on what specific skills they are missing relative to their peers.

2.2 Problem Statement

To design and develop an AI-powered system that automatically clusters student resumes based on their technical skill profiles, detects critical skill gaps between high-performing and low-performing groups, and generates personalized learning recommendations — all presented through an interactive, modern web dashboard.

2.3 Objectives

1. Build an end-to-end NLP pipeline to clean, tokenize, and process raw resume text.
2. Extract structured technical skills (programming languages, frameworks, tools, databases, cloud technologies) from unstructured resume text using a rule-based extraction engine.
3. Generate numerical feature vectors from text using TF-IDF vectorization.
4. Apply K-Means unsupervised clustering to segment candidates into distinct proficiency groups.
5. Perform comparative cluster analysis to identify the strongest and weakest student groups.
6. Detect critical skill gaps between clusters and generate targeted learning recommendations.
7. Develop a visually modern, interactive Streamlit dashboard for real-time data exploration and live resume analysis.

2.4 Scope

The system is designed for the academic placement ecosystem, targeting:

- Placement Cells: Automated bulk resume analysis and student categorization.
- Students: Self-assessment tool providing personalized skill gap feedback and learning paths.
- Faculty / Mentors: Data-driven overview of batch-level skill distribution.

3. Literature Survey

#	Title / Topic	Key Contribution	Relevance
1	TF-IDF for Document Representation (Salton & Buckley, 1988)	Introduced term weighting schemes for information retrieval.	Foundation of our text vectorization approach.
2	K-Means Clustering (MacQueen, 1967)	Proposed partitioning-based clustering using centroid minimization.	Core clustering algorithm used in our system.
3	NLTK: Natural Language Toolkit (Bird et al., 2009)	Comprehensive Python library for NLP tasks.	Used for tokenization, stopword removal, and lemmatization.

4	Resume Parsing Using NLP (Karan & Snajder, 2018)	Explored information extraction from resumes using NLP.	Inspired our rule-based skill extraction engine.
5	Silhouette Analysis for Cluster Validation (Rousseeuw, 1987)	Proposed silhouette coefficient for measuring cluster quality.	Used to evaluate our clustering performance.
6	Streamlit: Rapid ML App Development (Streamlit Inc., 2019)	Framework for building interactive ML dashboards in Python.	Used for our frontend dashboard.
7	Plotly for Interactive Visualization (Plotly Technologies, 2015)	Library for interactive, publication-quality charts.	Powers all charts and heatmaps in the dashboard.
8	Skill Gap Analysis in Education (World Economic Forum, 2023)	Highlighted the growing disconnect between academic curricula and industry needs.	Motivates the need for automated skill gap detection.

4. System Architecture

4.1 High-Level Architecture

The system follows a modular, sequential pipeline architecture as illustrated below:

```
Raw Resume Dataset (CSV) → Text Preprocessing → Skill Extraction Engine → TF-IDF Vectorization  
→ K-Means Clustering → Cluster Analysis & Skill Gap Detection → Recommendation Engine →  
Streamlit Dashboard
```

Additionally, PCA Dimensionality Reduction is applied post-clustering for 2D scatter plot visualization, feeding directly into the Streamlit Dashboard.

4.2 Project File Structure

```
SkillCluster_AI/  
├── datasets/Resume/Resume.csv      # Kaggle Resume Dataset (input)  
├── output/                          # Auto-generated model artifacts  
│   ├── processed_resumes.pkl       # Processed DataFrame with clusters  
│   ├── kmeans_model.pkl            # Trained K-Means model  
│   ├── tfidf_vectorizer.pkl        # Fitted TF-IDF vectorizer  
│   └── analysis_results.pkl         # Cluster analysis & skill gaps  
├── preprocessing.py                # Text cleaning pipeline  
├── extraction.py                   # Rule-based skill extraction  
├── embeddings.py                   # TF-IDF embedding generation  
├── clustering.py                   # K-Means & PCA/t-SNE  
├── analysis.py                     # Cluster evaluation & gap detection  
└── recommendation.py              # Learning path generator
```

```
|— pipeline.py           # End-to-end training pipeline
|— app.py               # Streamlit dashboard
|— requirements.txt     # Python dependencies
```

4.3 Data Flow (Level 1)

User/Dataset → Text Cleaner → Skill Extractor → TF-IDF Engine → K-Means Clusterer → Gap Analyzer → Dashboard / Reports. Intermediate artifacts (processed DataFrame, ML Models, Analysis Results) are persisted to storage and consumed by the dashboard.

5. Methodology

5.1 Data Collection

Parameter	Value
Source	Kaggle – Resume Dataset
Total Records	2,484 (full), 1,000 (sampled for training)
Key Columns	Resume_str (raw text), Category (job label)
Number of Job Categories	24
Format	CSV

5.2 Text Preprocessing

Raw resume text undergoes the following multi-stage cleaning pipeline:

Step	Technique	Purpose
1	Lowercasing	Normalize case for uniform matching
2	URL Removal	Strip http/https/www links
3	Email Removal	Remove email addresses
4	Special Character Removal	Keep only alphanumeric characters and spaces
5	Tokenization	Split text into individual word tokens (NLTK word_tokenize)
6	Stopword Removal	Remove English stopwords (NLTK corpus)
7	Lemmatization	Reduce words to root form using WordNetLemmatizer
8	Whitespace Normalization	Collapse multiple spaces into single spaces

Code implementation (preprocessing.py):

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'\S+@\S+', '', text)
    text = re.sub(r'[^a-z0-9\s]', ' ', text)
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return re.sub(r'\s+', ' ', ' '.join(tokens)).strip()
```

5.3 Skill Extraction Engine

A rule-based NLP extraction engine identifies technical skills from cleaned text by matching against a curated, multi-category skill dictionary:

Category	Example Skills	Total Keywords
Programming Languages	Python, Java, C++, JavaScript, SQL, R	18
Frameworks	React, Django, Flask, TensorFlow, Pandas	19
Tools	Git, Docker, Kubernetes, Jenkins, Linux	17
Databases	MySQL, PostgreSQL, MongoDB, Redis	11
Cloud & DevOps	AWS, Azure, GCP, Terraform, Ansible	13

The engine handles both single-word skills (e.g., python) and multi-word skills (e.g., ruby on rails, scikit-learn) using substring matching and word-set membership. A composite Total_Skills_Count feature is computed as the sum of all extracted skill categories per resume.

5.4 Feature Engineering — TF-IDF Vectorization

The cleaned resume text is transformed into numerical feature vectors using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization.

Mathematical Formulation: $TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$

- $TF(t, d)$ = Frequency of term t in document d

- $IDF(t) = \log(N / df(t))$, where N is total documents and $df(t)$ is the number of documents containing term t

Parameter	Value
Vectorizer	<code>sklearn.feature_extraction.text.TfidfVectorizer</code>
Max Features	1,000
Input	Cleaned resume text
Output	1000-dimensional sparse feature vector per resume

5.5 Unsupervised Clustering — K-Means

The TF-IDF feature matrix is fed into the K-Means clustering algorithm to partition resumes into K distinct groups.

Algorithm Steps:

8. Initialize K centroids randomly.
9. Assign each data point to the nearest centroid (Euclidean distance).
10. Recompute centroids as the mean of assigned points.
11. Repeat steps 2–3 until convergence.

Objective Function (WCSS): $J = \sum_i \sum_{(x \in C_i)} \|x - \mu_i\|^2$

Parameter	Value
Algorithm	K-Means (<code>sklearn.cluster.KMeans</code>)
Number of Clusters (K)	4
Initialization Runs (<code>n_init</code>)	10
Random State	42

5.6 Dimensionality Reduction — PCA

For 2D scatter plot visualization, Principal Component Analysis (PCA) is applied to reduce the 1000-dimensional TF-IDF vectors to 2 components, enabling intuitive cluster visualization in the dashboard.

5.7 Cluster Analysis & Skill Gap Detection

Post-clustering, each cluster is profiled by computing:

- Cluster Size: Number of resumes in each cluster.
- Average Skill Count: Mean of Total_Skills_Count per cluster.
- Dominant Skills: Top-10 most frequent technologies using collections.Counter.

The strongest cluster (highest avg skill count) and weakest cluster (lowest avg skill count) are identified. Skill gap detection compares the top-20 skills in the strongest cluster against the top-20 in the weakest, identifying skills present in the strong cluster but absent in the weak one.

5.8 Recommendation Engine

Missing skills are categorized into programming languages, tools, and cloud technologies, and context-aware learning path suggestions are generated:

- Missing core languages → "Learn fundamental programming languages..."
- Missing DevOps tools → "Add essential dev tools to your workflow..."
- Missing cloud skills → "Consider a beginner certification in AWS/Azure/GCP..."
- If gaps > 3 → Suggest building portfolio projects.

6. Implementation Details

6.1 Technology Stack

Component	Technology	Version / Details
Programming Language	Python	3.8+
NLP Library	NLTK	Tokenization, Stopwords, Lemmatization
ML Framework	Scikit-Learn	KMeans, PCA, TfidfVectorizer, Silhouette Score
Data Handling	Pandas, NumPy	DataFrame operations, numerical computation
Visualization	Plotly	Interactive charts, scatter plots, heatmaps
Web Framework	Streamlit	Dashboard UI with glassmorphic design
UI Enhancement	streamlit-option-menu	Horizontal navigation bar
Serialization	Joblib	Model persistence (.pkl files)

6.2 Pipeline Execution Flow

The end-to-end pipeline is orchestrated by `pipeline.py`, executing the following stages sequentially:

```
Step 1: Load CSV dataset
Step 2: Sample 1000 records (if dataset > 1000)
Step 3: Apply text preprocessing
Step 4: Extract structured skills
Step 5: Generate TF-IDF embeddings → Save vectorizer
Step 6: Apply K-Means clustering (K=4) → Save model
Step 7: Reduce dimensions via PCA → Add 2D coordinates
Step 8: Analyze clusters → Identify strongest/weakest
Step 9: Detect skill gaps
Step 10: Save all artifacts to output/ directory
```

6.3 Dashboard Pages

Page	Functionality
Dashboard	Displays total resumes, job categories, cluster count. Category pie chart and cluster bar chart.
Cluster Insights	Interactive PCA scatter plot of all resumes colored by cluster. Per-cluster skill profiles in tabbed view.
Skill Gaps	Strongest vs. weakest cluster comparison. Skill gap tags. Skill frequency heatmap.
Resume Analyzer	Live text input, real-time NLP extraction, cluster prediction, personalized learning recommendations.

6.4 UI Design

The dashboard employs a dark-mode glassmorphic design with:

- Custom CSS with backdrop-filter: blur(12px) for glass effect
- Gradient backgrounds with radial-gradient overlays
- Google Fonts: Outfit (headings) and Inter (body)
- Hover animations with transform: translateY(-8px) and glow effects
- Cyan (#00E5FF) and Purple (#7B2CBF) color palette

7. Experimental Results & Analysis

7.1 Dataset Statistics

Metric	Value
--------	-------

Total Resumes Processed	1,000
Unique Job Categories	24
Number of Clusters (K)	4
TF-IDF Feature Dimensions	1,000

7.2 Cluster Distribution

Cluster	Size	Avg. Skills Detected	Top 5 Dominant Skills
Cluster 0	166	1.11	Excel, C, SQL, Go, R
Cluster 1 (Strongest)	237	2.36	Excel, C, SQL, Linux, R
Cluster 2 (Weakest)	503	0.83	Excel, Chef, C, R, Go
Cluster 3	94	1.34	Excel, Oracle, R, C, SQL

Cluster 1 was identified as the strongest cluster with the highest average skill count of 2.36, indicating candidates with the richest technical profiles. Cluster 2 was identified as the weakest with an average of only 0.83 skills per resume.

7.3 Skill Gap Analysis Results

The following technologies were found in the strongest cluster's top skills but missing from the weakest cluster:

Missing Skill	Category
AWS	Cloud & DevOps
Git	Tools
MySQL	Databases
SQL Server	Databases
Bash	Tools
JIRA	Tools
MATLAB	Programming Languages
Bootstrap	Frameworks

These represent critical industry-demand skills that candidates in the weakest cluster should prioritize learning.

7.4 Model Evaluation — Silhouette Score

Metric	Value	Interpretation
Silhouette Score	0.0365	Weak mathematical separation, but expected for high-dimensional sparse TF-IDF vectors

Analysis of the Silhouette Score: A Silhouette Score of 0.0365 is mathematically low (scale: -1 to +1, where +1 = perfect separation). However, this result is standard and expected for the following reasons:

- **Curse of Dimensionality:** TF-IDF produces a 1000-dimensional sparse matrix where Euclidean distances between points converge, inherently deflating silhouette values.
- **Overlapping Vocabulary:** Resumes across different IT domains share substantial common vocabulary (e.g., "development", "team", "management"), causing cluster boundaries to blur.
- **Practical Validity:** Despite the low mathematical score, the clusters produce semantically interpretable and distinct skill profiles, which is the primary goal. Cluster 1 clearly separates candidates with richer technical stacks from the sparse-profile Cluster 2.

In published NLP clustering research, Silhouette Scores of 0.02–0.10 are commonly reported for TF-IDF + K-Means on document datasets. Our score of 0.0365 falls within this expected range.

7.5 Category Distribution (Top 10)

Job Category	Resume Count
Information Technology	59
Construction	56
Chef	53
Advocate	53
Sales	51
Teacher	48
Business Development	48

Banking	48
Accountant	45
Fitness	45

8. Dashboard & Visualization

The Streamlit dashboard provides four interactive views:

8.1 Dashboard (Overview Page)

- Three glassmorphic metric cards displaying: Total Resumes (1,000), Job Categories (24), Clusters Formed (4).
- Donut chart showing category distribution across all resumes.
- Bar chart showing the number of resumes per cluster.

8.2 Cluster Insights Page

- PCA Scatter Plot: Interactive 2D projection of all 1,000 resumes, color-coded by cluster assignment. Hover tooltips show job category and skill count.
- Cluster Skill Profiles (Tabbed): Each tab displays the average skill count and dominant technologies for that cluster as styled tags.

8.3 Skill Gaps Page

- Side-by-side comparison of the strongest (Cluster 1) and weakest (Cluster 2) groups.
- Skill gap tags highlighted in red, showing missing technologies.
- Skill Frequency Heatmap: A Plotly heatmap comparing the frequency of top skills between strong and weak clusters.

8.4 Resume Analyzer Page

- Text input area for pasting raw resume text.
- On submission: performs real-time NLP extraction → TF-IDF vectorization → K-Means cluster prediction → displays detected skills and AI-generated learning path recommendations.

9. Conclusion

SkillCluster AI successfully demonstrates the application of NLP and Unsupervised Machine Learning techniques for automated resume analysis and skill gap detection. The key achievements of this project are:

12. **Built a robust, multi-stage text processing pipeline that transforms raw, unstructured resume text into structured, machine-readable skill profiles using NLTK and regex-based extraction.** Automated NLP Pipeline:
13. **The K-Means algorithm (K=4) successfully segments 1,000 resumes into 4 distinct proficiency tiers, with Cluster 1 (strongest, avg. 2.36 skills) clearly separating from Cluster 2 (weakest, avg. 0.83 skills).** Effective Clustering:
14. **The system identified 8 critical skill gaps (AWS, Git, MySQL, Bash, JIRA, etc.) that students in the weakest cluster are missing compared to the strongest — providing concrete, data-driven improvement targets.** Actionable Skill Gap Detection:
15. **The recommendation engine generates context-aware, categorized learning paths (languages, tools, cloud certifications, portfolio projects) tailored to each individual's gap profile.** Personalized Recommendations:
16. **A modern, glassmorphic Streamlit application with interactive Plotly visualizations enables real-time exploration of cluster data and live resume analysis for individual candidates.** Production-Quality Dashboard:

The project validates that even with baseline NLP techniques (TF-IDF) and traditional clustering (K-Means), meaningful and interpretable insights can be extracted from resume data — insights that are directly actionable for students, placement cells, and academic institutions.

10. Future Scope

Enhancement	Description
Transformer Embeddings	Replace TF-IDF with BERT / Sentence-Transformers for semantic-aware, dense embeddings that capture deeper contextual meaning.
Multi-Format Resume Parsing	Add PDF and DOCX parsing (using PyPDF2, python-docx) to accept file uploads directly.
Dynamic K Selection	Implement Elbow Method or Gap Statistic for automatic optimal K selection.
DBSCAN / HDBSCAN	Explore density-based clustering to handle non-spherical cluster shapes and noise.
Industry Skills API	Integrate with live job posting APIs (LinkedIn, Indeed) to dynamically update the skill dictionary.
LLM-Powered Recommendations	Use GPT / Gemini APIs to generate more nuanced, natural-language learning roadmaps.

User Authentication	Add login system for students to track their skill growth over time.
Batch Export	Generate downloadable PDF reports for placement cell coordinators.

11. References

17. Salton, G. & Buckley, C. (1988). "Term-Weighting Approaches in Automatic Text Retrieval." *Information Processing & Management*, 24(5), 513–523.
18. MacQueen, J. (1967). "Some Methods for Classification and Analysis of Multivariate Observations." *Proceedings of 5th Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, 281–297.
19. Rousseeuw, P. J. (1987). "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis." *Journal of Computational and Applied Mathematics*, 20, 53–65.
20. Bird, S., Klein, E. & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
21. Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2825–2830.
22. Karan, M. & Snajder, J. (2018). "Cross-Domain Detection of Abusive Language Online." *Proceedings of the 2nd Workshop on Abusive Language Online*, ACL.
23. World Economic Forum (2023). *Future of Jobs Report 2023*. Geneva, Switzerland.
24. Streamlit Inc. (2019). Streamlit — The fastest way to build and share data apps. <https://streamlit.io/>
25. Plotly Technologies Inc. (2015). Collaborative data science. <https://plotly.com/>
26. NLTK Project (2024). Natural Language Toolkit. <https://www.nltk.org/>

End of Report